IS Elec 2

Web Systems and Technologies

Phase 2 Backend Development and API Integration

Lab Report 4

User and Product Management API

Group Phase 2 Backend Team

Member	Role
Bea Roshan Pabilona	Project Manager and Documentation Officer
Christian Balauro	Backend Developer
Simranjit Singh	Database Manager
Karl James Matias	GitHub Manager using kjamesss

GitHub Manager account kjamesss

Date 13 November 2026

This report documents the backend setup, Mongoose models, REST API routes, testing results, and GitHub work for Phase 2. The API passed 37 integration tests and all 52 assertions in the 20 request Postman collection.

The phase title corrects the Phase 3 typo in the activity handout. The console demonstration shows the required backend through real HTTP requests.

Backend setup and local demonstration

The backend uses Node.js and Express to accept JSON requests. Mongoose validates and stores users and products in MongoDB. Each part has its own folder, which makes the code easier to review and extend.

```
backend/
  server.js          startup and shutdown
  app.js             middleware and route mounting
  config/            MongoDB connection
  models/            User and Product schemas
  controllers/       reusable CRUD and password hashing
  routes/            user and product endpoints
  middleware/        validation and error responses
  scripts/           local database, demo, Postman runner
  tests/             integration tests
```

Run the backend

```
cd backend
npm ci
Copy-Item .env.example .env
# Terminal 1
npm run db:local
# Terminal 2, inside backend
npm start
# Terminal 3, inside backend
npm run demo
```

The local database launcher runs a real MongoDB executable with WiredTiger storage. Development data is kept under backend/.local-db. An existing local MongoDB server or Atlas connection can also be used. The first launcher run needs internet access.

The required start command uses nodemon server.js. Express accepts requests after MongoDB connects and the model indexes are ready. Run npm run demo in a third terminal to show HTTP requests and JSON responses.

Environment variable	Local value
PORT	5000
MONGO_URI	mongodb://127.0.0.1:27017/phase2_backend
CORS_ORIGIN	http://localhost:3000

The local .env file is ignored by Git. The committed .env.example lets group members configure their own machine without publishing credentials. On macOS or Linux, use cp .env.example .env.

Server and database connection code

The server loads environment settings, connects to MongoDB, and starts the Express application. HTTP middleware and route mounting are in app.js so tests can use the application directly.

server.js

```
require('dotenv').config({ path: require('node:path').join(__dirname, '.env') });
const mongoose = require('mongoose');
const connectDB = require('./config/db');
const app = require('./app');

async function start() {
  await connectDB();
  const port = Number(process.env.PORT || 5000);
  const server = app.listen(port, '127.0.0.1', () => {
    console.log(`Server running on port ${port}`);
  });
  server.on('error', async (error) => {
    console.error(`Server error: ${error.code || 'unknown'}`);
    await mongoose.disconnect();
    process.exitCode = 1;
  });
  const shutdown = () => server.close(async () => {
    await mongoose.disconnect();
    process.exit(0);
  });
  process.once('SIGINT', shutdown);
  process.once('SIGTERM', shutdown);
}

start().catch(async (error) => {
  console.error(error.message.startsWith('MONGO_URI')
    ? error.message : 'Startup failed. Check MongoDB availability and backend/.env.');
  await mongoose.disconnect();
  process.exitCode = 1;
});
```

config db.js

```
const mongoose = require('mongoose');

async function connectDB() {
  if (!process.env.MONGO_URI) throw new Error('MONGO_URI is required in backend/.env');
  await mongoose.connect(process.env.MONGO_URI, { serverSelectionTimeoutMS: 5000 });
  await Promise.all([
    require('../models/User').init(),
    require('../models/Product').init()
  ]);
  console.log('MongoDB Connected');
}

module.exports = connectDB;
```

Model initialization waits for the indexes, including the unique email index. A failed database connection prevents startup. Graceful shutdown closes the HTTP server and the Mongoose connection.

Sample model and user routes

User model

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true, minlength: 2, maxlength: 80 },
  email: {
    type: String, required: true, unique: true, lowercase: true, trim: true,
    maxlength: 254, match: /^[^\s@]+@[^\s@]+\.[^\s@]+$/
  },
  password: { type: String, required: true, select: false },
  role: { type: String, enum: ['user', 'admin'], default: 'user' }
}, { timestamps: true, strict: 'throw' });

userSchema.set('toJSON', {
  transform: (doc, ret) => { delete ret.password; delete ret.__v; return ret; }
});

module.exports = mongoose.model('User', userSchema);
```

The model validates the user name, email address, and role. The controller hashes the password with scrypt and a random salt before saving it. Passwords are excluded from JSON output. Product data has its own model with nonnegative price and integer stock validation.

User routes

```
const router = require('express').Router();
const controller = require('../controllers/userController');
const { validateId, validateBody } = require('../middleware/validate');
const fields = ['name', 'email', 'password', 'role'];

router.route('/')
  .post(validateBody(fields, ['name', 'email', 'password']), controller.create)
  .get(controller.list);
router.route('/:id')
  .all(validateId)
  .get(controller.get)
  .put(validateBody(fields), controller.update)
  .delete(controller.remove);

module.exports = router;
```

The router selects a controller based on the HTTP method. Middleware checks the ID and request fields first. The controller performs the Mongoose operation and returns JSON. Product routes use the same pattern under /api/products.

Route mounting in app.js

```
app.use('/api/users', userRoutes);
app.use('/api/products', productRoutes);
```

PUT changes the editable fields supplied in the request and preserves omitted fields. The API returns 400 for invalid input, 404 for a missing resource, and 409 for a duplicate email address.

Actual backend output

These responses were read from the completed npm run demo output. The update, deletion, and final 404 response use the same user ID.

```
GET /api/health
HTTP 200
{
  "status": "ok",
  "database": "connected"
}

POST /api/users
HTTP 201
{
  "name": "Demo User",
  "email": "demo.1794560968420@example.test",
  "role": "user",
  "_id": "6af6d3c802ce658be0889dae",
  "createdAt": "2026-11-13T09:09:28.478Z",
  "updatedAt": "2026-11-13T09:09:28.478Z"
}

PUT /api/users/6af6d3c802ce658be0889dae
HTTP 200
{
  "_id": "6af6d3c802ce658be0889dae",
  "name": "Updated Demo User",
  "email": "demo.1794560968420@example.test",
  "role": "user",
  "createdAt": "2026-11-13T09:09:28.478Z",
  "updatedAt": "2026-11-13T09:09:28.501Z"
}

DELETE /api/users/6af6d3c802ce658be0889dae
HTTP 200
{
  "message": "User deleted",
  "id": "6af6d3c802ce658be0889dae"
}

GET /api/users/6af6d3c802ce658be0889dae
HTTP 404
{
  "message": "User not found"
}
```

The product demonstration followed the same sequence: create 201, read 200, update 200, delete 200, and read after deletion 404. The complete console transcript and parsed results are saved in docs/evidence.

Backend output and testing results

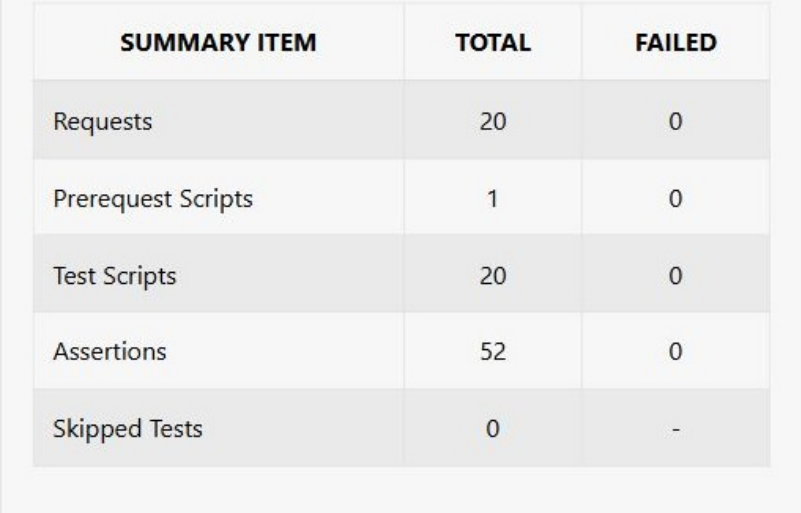
Run npm run demo from backend after starting MongoDB and Express. The script creates, reads, updates, and deletes records through actual HTTP requests. The captured output is in docs/evidence/backend-demo.txt.

Operation	User resource	Product resource	Status
Create	POST /api/users	POST /api/products	201
Read	GET /api/users/:id	GET /api/products/:id	200
Update	PUT /api/users/:id	PUT /api/products/:id	200
Delete	DELETE /api/users/:id	DELETE /api/products/:id	200
Confirm deleted	GET same ID	GET same ID	404

The Postman collection was executed with Newman, Postman's command-line collection runner. The screenshots show the generated request and response report. The collection and local environment are included for import into Postman.

Verification	Passed	Failed
Database-backed integration tests	37	0
Postman requests	20	0
Postman assertions	52	0

The tests also check persisted updates, duplicate emails, password protection, malformed IDs, invalid JSON, pagination, and database health. The runtime dependency audit reported zero known vulnerabilities on the test date.



SUMMARY ITEM	TOTAL	FAILED
Requests	20	0
Prerequest Scripts	1	0
Test Scripts	20	0
Assertions	52	0
Skipped Tests	0	-

Figure 1. Summary table captured from the completed Postman collection run.

Create a user

POST /api/users returned 201 Created. The response contains the saved ID and excludes the password.



Figure 2. Actual request and JSON response from the local API.

Retrieve users

GET /api/users returned 200 OK and included the user created in the preceding request.

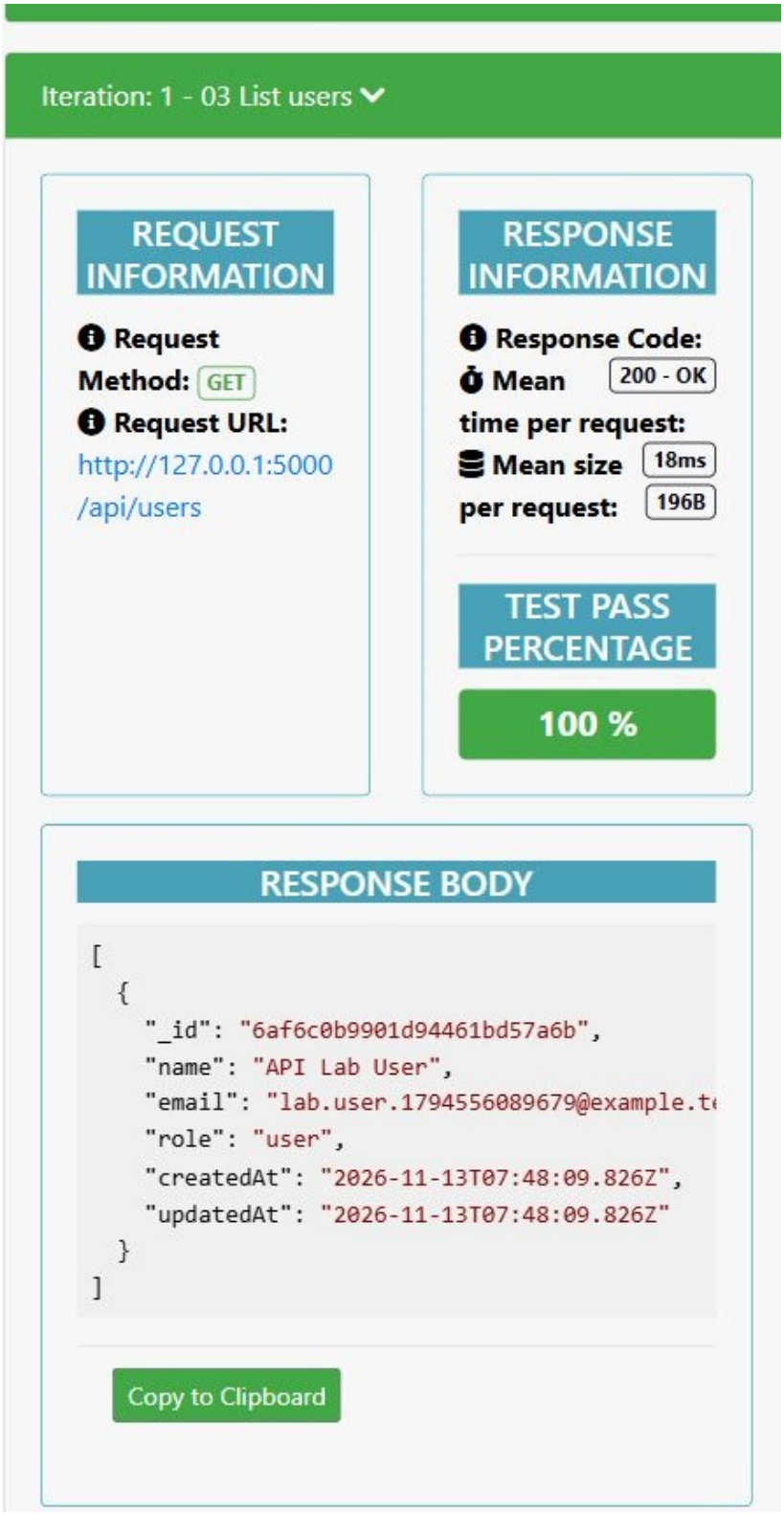


Figure 3. Actual request and JSON response from the local API.

Update a user

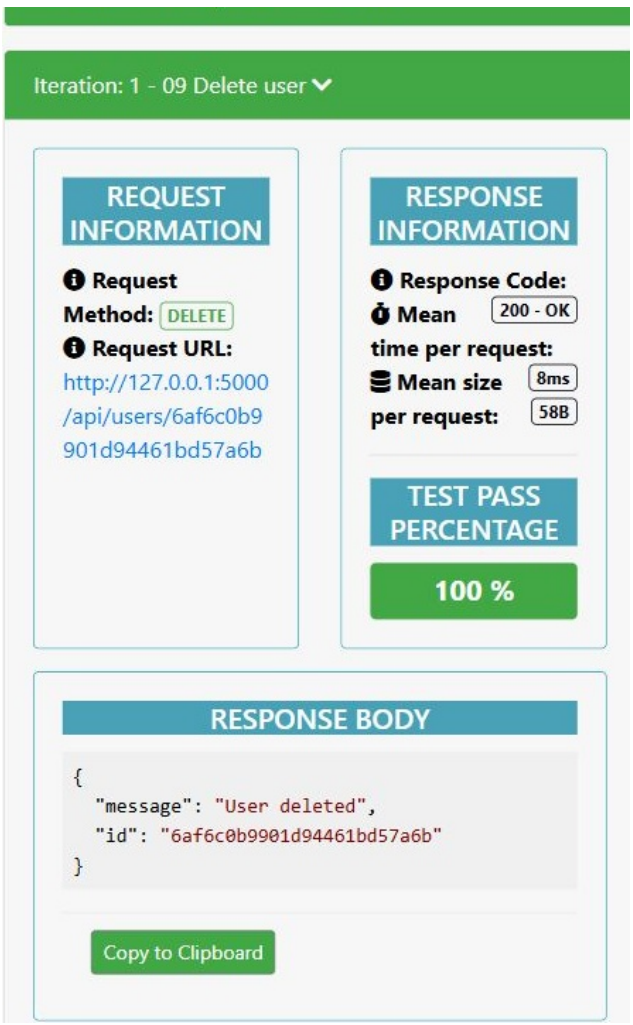
PUT /api/users/:id returned 200 OK with the name Updated Lab User. A separate GET request confirmed that the change persisted.

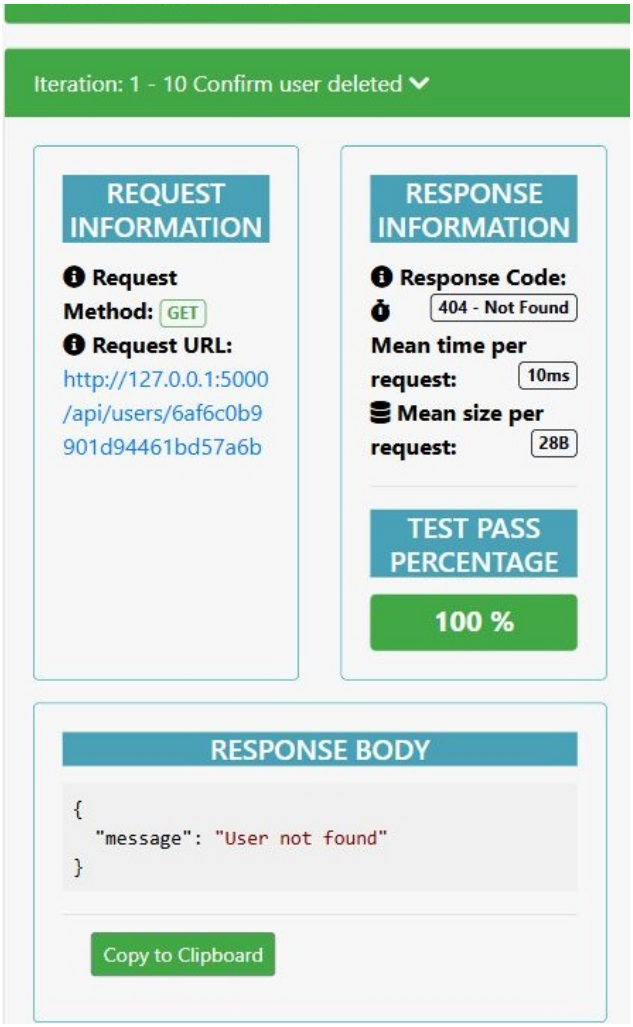


Figure 4. Actual request and JSON response from the local API.

Delete the user and confirm the result

DELETE /api/users/:id returned 200 OK with a deletion message and the matching user ID. GET for the same ID returned 404 Not Found after deletion.





Figures 5 and 6. The deletion response and the follow-up 404 response use the same user ID.

These original API screenshots are preserved with their matching raw run in api-screenshot-results.json. The fresh Postman run also passed all 20 requests and 52 assertions.

Products passed the same CRUD sequence. Their screenshots are saved as 07-product-create.jpg through 10-product-delete.jpg in docs/evidence. The Markdown report includes every product screenshot. The Postman collection also confirms product deletion with a 404 response.

Group contributions and GitHub commits

The assigned roles below identify each member's responsibility for reviewing and maintaining the submitted work.

Bea Roshan Pabilona

Project management and documentation. Coordinates requirements, checks the report against the activity, and maintains the cover page, setup explanation, screenshots, and contribution record.

Christian Balauro

Backend development. Owns Express setup, user and product routes, reusable CRUD controllers, request validation, and the backend demonstration script.

Simranjit Singh

Database management. Owns the MongoDB connection, User and Product schemas, field validation, unique email index, and verification that updates and deletion persist.

Karl James Matias

GitHub management. Owns repository organization and clear commit messages. GitHub access and the submitted commits use the kjamesss account.

Verified GitHub commits

3b5c316 Build modular Express backend with MongoDB models and complete CRUD routes

2e5ade4 Test user and product APIs with integration tests and a Postman collection

f4afc31 Record successful API tests and add a backend-only console demonstration

16991cd Add Phase 2 lab report with code snippets screenshots and rubric evidence

c3c0e7a Preserve report and screenshot bytes across operating systems

306bc5b Restore required backend scope and console demonstration

The implementation and testing evidence were pushed through the authorized kjamesss account. The report and checklist are included in a subsequent documentation commit.

Repository on GitHub | Complete commit history

Rubric evidence checklist

Each category is worth 10 points. The work below addresses the top descriptor in all five categories. The final score is assigned by the instructor.

Backend Architecture Setup

Modular and scalable. server.js, app.js, config, models, controllers, routes, and middleware separate responsibilities.

MongoDB and Mongoose Integration

Connected and working. Actual MongoDB process, two Mongoose models, health response, and database-backed persistence tests.

REST API Route Functionality

CRUD endpoints tested. User and product CRUD pass in the Postman collection. The saved run has 20 requests and 52 passing assertions.

GitHub Commit and Documentation

Clear commits and README present. Pushed commits, root README, backend README, environment template, and committed report and evidence.

Lab Report Quality

Clear and well organized. Cover page, setup overview, exact code snippets, real testing screenshots, commit references, and member responsibilities.

Reproduce the recorded checks

```
cd backend
npm test
# With MongoDB and Express already running
npm run test:postman
npm run demo
```

The evidence folder contains the integration test log, Newman JSON results, text run output, request screenshots, and the backend demonstration transcript. The Postman report uses a light presentation. Development-only Newman dependencies have upstream audit findings and are used for local testing.

References

Supplied IS Elec 2 activity handout and Lab Report 4 rubric, with the Phase 2 title correction.

Express routing

Mongoose schemas

Postman Newman documentation